

CIC0101 - Sistemas de Informação

Aula 10 - Desenvolvimento de Sistemas de Informação

Profa. Patrícia Araújo de Oliveira

Universidade de Brasília (UnB) - Departamento de Ciência da Computação

2026.2

De onde estamos partindo?

O desenvolvimento começa antes da primeira linha de código.



O próximo passo é transformar esse diagnóstico em uma definição consistente do que precisa mudar — antes de decidir como construir.

Sistema de Informação não é sinônimo de software

Essa distinção conecta SI e Engenharia de Software.

Sistema de Informação

Sistema sociotécnico: pessoas, processos, dados/informações e tecnologia articulados para apoiar atividades e objetivos.

Software

Componente tecnológico que implementa funções computacionais e pode integrar um SI.

Engenharia de Software

Disciplina que fornece processos, princípios, métodos e práticas para desenvolver e evoluir software de forma sistemática.

Desenvolver bem o software é necessário — mas não garante, sozinho, que o SI resolva o problema organizacional.

A ponte entre as duas áreas

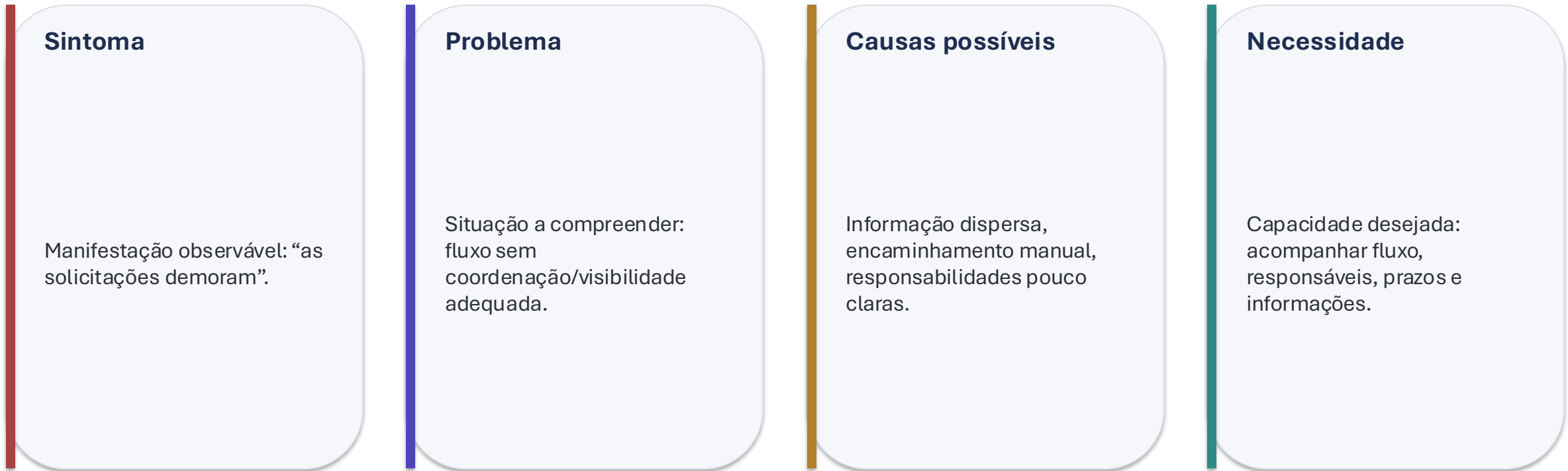
O problema continua sendo organizacional; a ES ajuda a construir parte da resposta.



Não vamos transformar CIC0101 em Engenharia de Software: vamos usar conceitos de ES onde eles explicam como a solução é desenvolvida.

Problema, sintoma, causa e necessidade

Antes de especificar software, precisamos investigar.



Uma causa é hipótese até ser sustentada por evidências do contexto.

Caso — “Cadê o projetor?”

Antes de pensar em sistema, vamos entender o que aconteceu.

14h — começa a aula

A professora chega ao laboratório e o projetor portátil que costuma ficar no armário não está lá.

A primeira busca

O técnico lembra que alguém retirou o equipamento, mas não sabe quem. Há uma planilha de empréstimos em outro computador — e nem toda retirada é registrada.

A informação aparece

Depois de mensagens no grupo, descobrem que um aluno levou o projetor na sexta. Ele diz que avisou a um monitor e que devolveria na segunda.

Mas o controle continua incerto

O monitor confirma a conversa, porém ninguém atualizou a planilha. Quando o projetor aparece, ainda não está claro quem é responsável por ele naquele momento.

Alguém conclui: “A gente precisa de um sistema para gerenciar os equipamentos.”

Antes de propor a solução: o que ainda não sabemos?

A história foi deixada incompleta de propósito.

Processo

Quem pode retirar? Quem autoriza? Como deveria ocorrer empréstimo e devolução?

Responsabilidades

O monitor podia autorizar? Quem responde pelo equipamento enquanto ele está fora?

Informações

Que registros existem? Onde ficam? Quem atualiza? Quais informações são realmente necessárias?

Regras e exceções

Todos os equipamentos podem sair? Existe termo? E manutenção, atraso, perda ou devolução parcial?

Desenvolver um SI começa também por descobrir aquilo que ainda não sabemos.

Atividade — investigando o caso antes da solução

5–7 minutos

A frase abaixo surgiu no fim da história. Agora desmontem-na.

“O problema é que não existe um sistema para gerenciar os equipamentos do laboratório.”

Sintomas

O que acontece hoje?

Causas

Por que pode acontecer?

Necessidade

O que precisa melhorar?

Evidências

O que precisamos verificar?

Regra: ainda não proponham funcionalidades.

Então, qual é o problema?

Sintomas

Equipamentos difíceis de localizar;
não se sabe quem usa; informações
dispersas ou desatualizadas.

Causas — hipóteses

Registros descentralizados; processo
pouco definido; atualização
inconsistente.

Necessidade

Melhorar controle e rastreabilidade;
conhecer situação, localização e uso.

Evidências

Como é controlado hoje? Quem
registra? Onde ocorrem erros,
atrasos e retrabalho?

Solução disfarçada de problema

“Não existe um sistema...”

Problema reformulado

“Há dificuldade para controlar e rastrear a localização, a situação e o uso dos equipamentos.”

O que a Engenharia de Software chama de processo de software?

Uma estrutura para organizar o trabalho de produção e evolução do software.

Processo de software = conjunto organizado de atividades para especificar, desenvolver, validar e evoluir software.

Especificação

o que deve ser feito e sob quais restrições

Desenvolvimento

projeto/design + implementação

Validação

o produto atende ao que é necessário?

Evolução

adaptar o software a novas necessidades

Essas atividades fundamentais aparecem em Sommerville; processos concretos podem organizá-las de formas diferentes.

Engenharia de Software: uma lente para o desenvolvimento

Não estamos transformando SI em uma disciplina de ES — estamos usando ES para estruturar o desenvolvimento do componente de software.

Sistema de Informação

Pessoas, processos, dados/informação, tecnologia e contexto organizacional.

Software

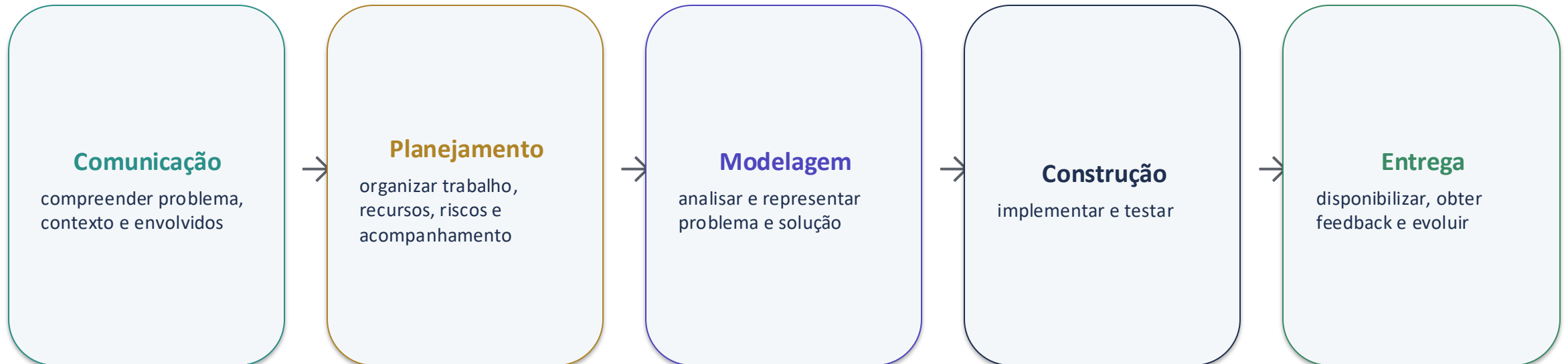
Um componente tecnológico que pode apoiar e automatizar partes do SI.

Engenharia de Software

Processos, princípios, métodos e práticas para desenvolver, entregar e evoluir software.

O desenvolvimento precisa ser organizado

Pressman & Maxim: cinco atividades metodológicas genéricas.



Essas atividades descrevem o trabalho — não obrigam uma única ordem rígida de execução.

Processo de software: atividades + organização no tempo

Um processo define como o trabalho de desenvolvimento será conduzido e coordenado.

Atividades

O que precisa ser feito? Comunicação, planejamento, modelagem, construção, entrega e atividades de apoio.

Fluxo

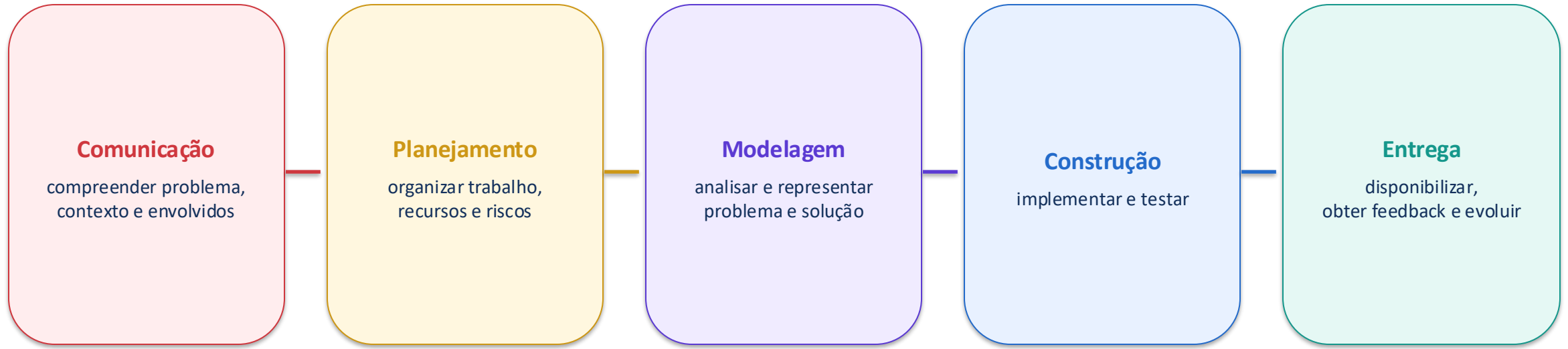
Em que ordem? O que pode ocorrer em paralelo? O que será revisitado?

Artefatos + controle

O que será produzido, revisado, testado, acompanhado e alterado?

Um modelo de referência: Pressman

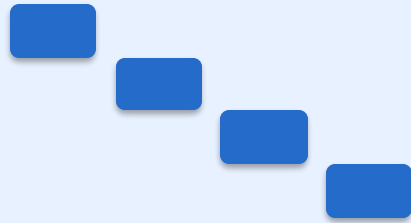
As atividades metodológicas organizam o trabalho — sem impor uma sequência rígida.



Comunicação, planejamento, modelagem, construção e entrega podem se repetir e se sobrepor ao longo do projeto.

Alguns modelos de processo de desenvolvimento

As mesmas atividades podem ser organizadas de maneiras diferentes.



Linear

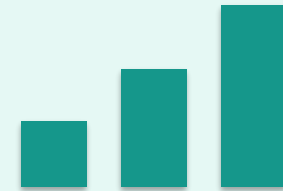
Predomínio de sequência entre atividades.

Mais adequado quando há maior estabilidade e previsibilidade.



Iterativo

Repetimos atividades, revisamos o que foi feito e incorporamos feedback e aprendizado.



Incremental

O produto cresce por partes. Cada incremento acrescenta capacidade e produz novas evidências.

Por que não definir tudo no início?

- Problemas organizacionais nem sempre são completamente compreendidos no começo.
- Stakeholders podem ter perspectivas, prioridades e necessidades diferentes.
- Necessidades mudam quando o contexto muda — e também quando uma solução começa a ser usada.
- Feedback reduz o risco de desenvolver uma solução tecnicamente correta para o problema errado.
- Por isso, modelos incrementais e iterativos são úteis quando há incerteza e necessidade de aprendizado.

Aplicando ao caso dos equipamentos

Imagine que decidíssemos desenvolver a solução em um único grande ciclo.

Antes de desenvolver

O que ainda não sabemos sobre empréstimos, devoluções, manutenção, responsáveis, localização e informações existentes?

Durante o desenvolvimento

O que aconteceria se descobríssemos que laboratórios diferentes trabalham de formas diferentes?

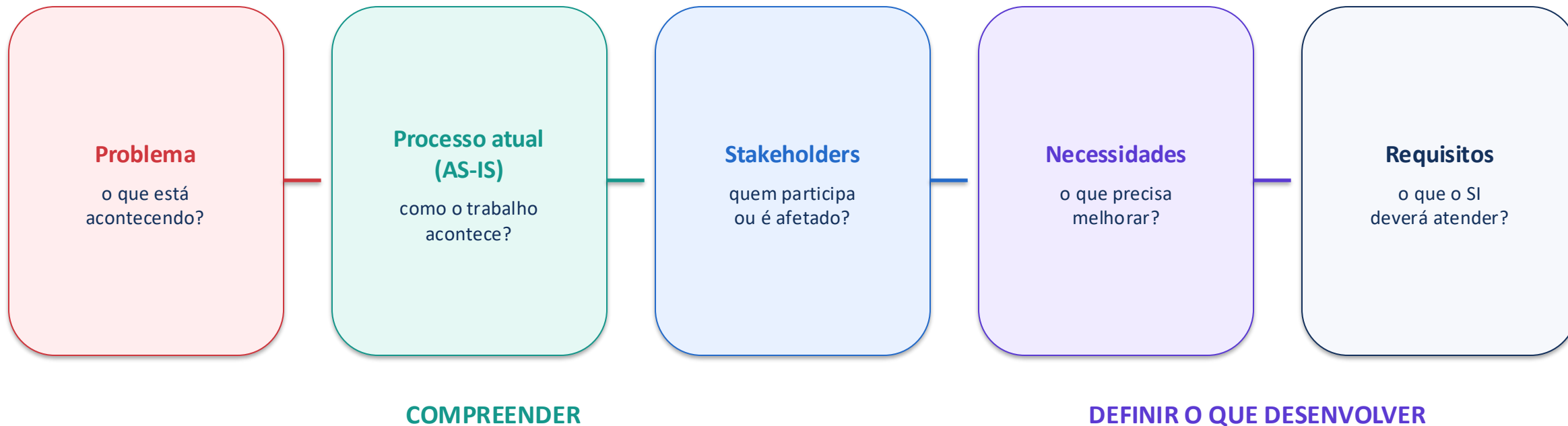
Após a primeira entrega

Que feedback poderia obrigar a revisar processo, necessidades, requisitos ou interface?

Qual seria o risco de “fechar tudo” antes de conversar com quem participa do processo?

O desenvolvimento começa produzindo conhecimento

Antes de especificar a solução, precisamos reduzir incerteza sobre o problema.



Hoje paramos antes da especificação detalhada de requisitos.

Comunicação não é apenas perguntar “o que você quer?”

Em Engenharia de Software, compreender requisitos exige interação com stakeholders e compreensão do domínio.

Descobrir

Conversar, observar, analisar documentos e compreender o domínio e o processo existente.

Compreender perspectivas

Diferentes stakeholders podem descrever o mesmo processo de maneiras distintas — e até conflitantes.

Organizar o conhecimento

Histórias, cenários, modelos de processo e outras representações ajudam a tornar explícito o que foi aprendido.

Quem é stakeholder?

Pessoa, grupo ou organização que participa, influencia, fornece informação, utiliza resultados ou é afetado pelo sistema e pelo processo.

Executa

realiza atividades do processo

Decide

autoriza, prioriza ou resolve exceções

Fornece / usa informação

produz dados ou depende das informações geradas

É afetado

sofre consequências do processo ou da solução

Atividade — quem precisamos ouvir no caso dos equipamentos?

- Liste pelo menos 4 stakeholders relevantes.
- Para cada um, indique: qual papel exerce no processo? que informação possui? que problema percebe?
- Identifique pelo menos um possível conflito de perspectiva.
- Escreva 3 perguntas que você faria antes de definir qualquer requisito do sistema.

Exemplo de tensão: quem empresta quer rapidez; quem responde pelo patrimônio pode priorizar rastreabilidade e controle.

Do stakeholder à necessidade — ainda não ao requisito

Stakeholder

“Sou responsável pelo laboratório e preciso saber onde estão os equipamentos.”

Necessidade

Ter rastreabilidade sobre localização, responsável, situação e movimentações.

Requisito — depois

A necessidade será traduzida e documentada em requisitos na próxima etapa da disciplina.

Não confundir necessidade com a primeira funcionalidade que nos vem à cabeça.

E como enxergar melhor o processo atual?

HOJE

Representamos o AS-IS com caixas, setas, decisões e informações suficientes para discutir o problema.

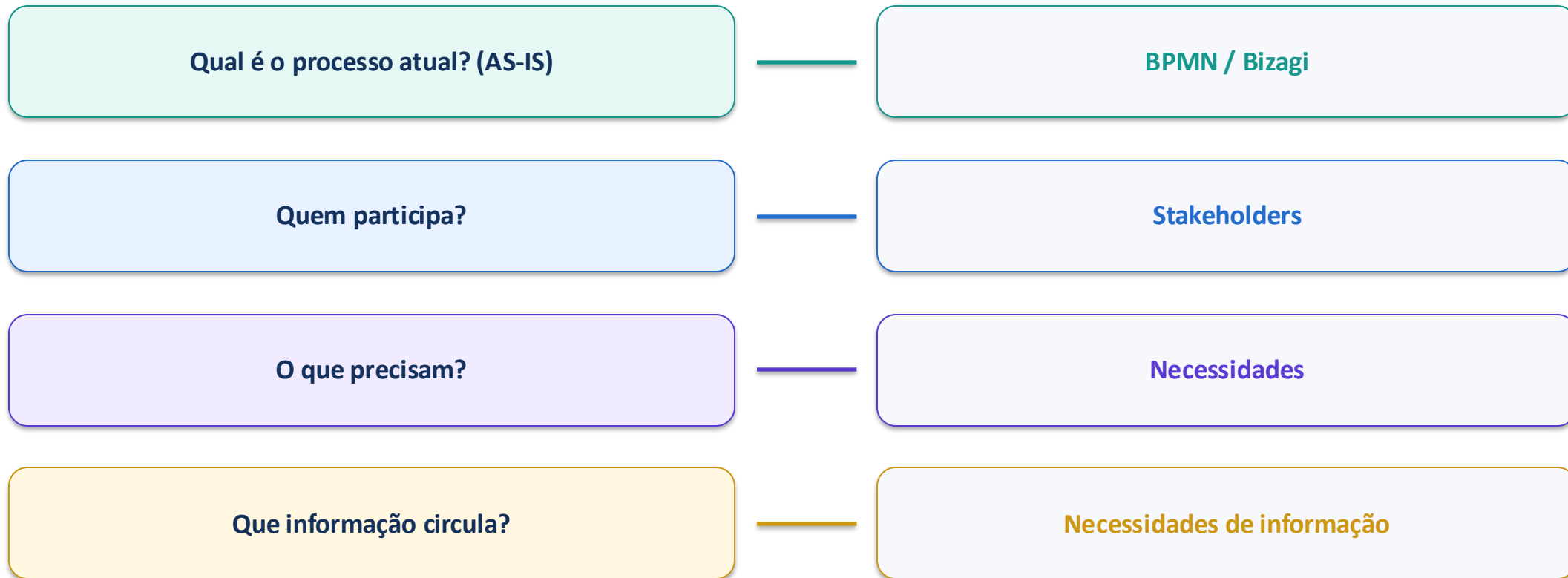
PRÓXIMA AULA

Vamos tornar a representação mais precisa com BPMN e usar o Bizagi Modeler como ferramenta.

A ferramenta não é o objetivo: o modelo servirá para descobrir atores, decisões, informações, gargalos e necessidades.

Próxima aula

01/10 • Processos, stakeholders e necessidades



Referências

- PRESSMAN, R. S.; MAXIM, B. R. Engenharia de Software: uma abordagem profissional. 9. ed., 2021.
- SOMMERVILLE, I. Software Engineering. 10th ed. Pearson, 2016.
- Material didático: Aspectos Importantes da Engenharia de Software — capítulos 1–4 de Pressman & Maxim.
- Material didático: Elicitação de Requisitos — stakeholders, descoberta/compreensão, histórias e cenários.
- ISO/IEC/IEEE 12207:2026 — referência normativa complementar sobre processos do ciclo de vida de software.